



UNIVERSIDAD DE LOS LAGOS

TÍTULO

SUBTÍTULO

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA

ASIGNATURA
CAMPUS ???, CHILE

Autor
Autor
Autor
Autor

31 de mayo de 2026

Índice general

1. Introducción	1
2. Desarrollo	2
2.1. Construcción de imágenes	2
2.2. Ejecución de contenedores	3
2.3. Verificación de contenedores activos	3
2.4. Verificación de logs y red	3
2.5. Prueba funcional	4
3. Conclusión	5

Índice de figuras

2.1. docker compose build	2
2.2. docker compose up -d	3
2.3. docker ps	3
2.4. Caption	3
2.5. Descripción de ambas imágenes.	4

Capítulo 1

Introducción

En el presente informe, se realizará la implementación de una aplicación web mediante el uso de **Docker**. El propósito central de este trabajo es desplegar un entorno de desarrollo que incluya tanto un servidor web con PHP como una base de datos MySQL, utilizando **Docker Compose** para gestionar ambos servicios de manera integrada.

A lo largo del documento, se describirá cómo se configuró la comunicación entre estos componentes, asegurando que la aplicación sea capaz de ejecutar operaciones **CRUD** (crear, leer, modificar y borrar datos) y que la información almacenada se mantenga persistente. Finalmente, mostraremos las evidencias que confirman que el sistema funciona correctamente.

Capítulo 2

Desarrollo

En esta sección se detalla el proceso de despliegue de la aplicación web utilizando **Docker** y **Docker Compose**.

2.1. Construcción de imágenes

El primer paso consistió en la compilación de la imagen base de la aplicación. Mediante el comando `docker compose build`, se ejecuto el Dockerfile configurado, el cual instala las dependencias necesarias de **PHP** para la conexión a la base de datos.

```
root@debian:~/taller-docker# docker compose build
[*] Building 4508.9s (10/10) FINISHED
-> [internal] load local definitions
-> -- reading from stdin 4908
-> [internal] load build definition from Dockerfile
-> [internal] load metadata for docker.io/library/php:8.2-apache
-> [internal] load .dockerignore
-> -- transferring context 0
-> [1/3] FROM docker.io/library/php:8.2-apache@sha256:affc043f8b9a9a5a6394a71d1d2726fcb4e6dca040a3b94f925b6138858dd
-> resolve docker.io/library/php:8.2-apache@sha256:affc043f8b9a9a5a6394a71d1d2726fcb4e6dca040a3b94f925b6138858dd
-> sha256:759c36286c4f08480ff49f674c06095ca61670ff20804960923c811ca 8928 / 8928
-> sha256:4f40b706bf54461c4a0271aeb0b9d61eb0d557708a6d7508dc308ac1 329 / 329
-> sha256:f16ef03632036203fc595a08a22974f85e091fad06409942e0880029e85f 2498 / 2498
-> sha256:3a2c3e28c7f6a80ff4158092146c83040770376705c36a2767260400400a 2558 / 2558
-> sha256:11d11022f6a0b1c5e0a0d45303a109011312b0ca3e07b00690611f7 2 4048 / 2 4048
-> sha256:e084c55a5a8d21149f0bc67d11ae38892d2150aac098a069cda0b29188 113 4048 / 11 4048
-> sha256:e4805c58575c11bad1c29f653759f6195a5c16174e240077af940465211 4888 / 4888
-> sha256:72260a0a0b10c7c2080530a23c1eac7293392990a54a5c415120a0f8 124 1390 / 124 1390
-> sha256:3c347adda7999af310fbb09f621ed071802e96223efbba049f54f0f4a09e 4888 / 4888
-> sha256:00c72157fac180bc34216a58c420275de07675001157c642b0b0c7005e5 4358 / 4358
-> sha256:f28f7780f7a03c16a5805908700f0af77a0b0c52437c958c463132a6e 4 2190 / 4 2190
-> sha256:5c080f5f90a76761345407c08c5e1da28abc784762ca62156638a089f25b 2268 / 2268
-> sha256:487697150f0056a29521874e757ab1114f0ea30154e702a58c1a6e53f39 117 9490 / 117 9490
-> sha256:55a48f952f4e34451107753e544c95540c407103750618410acac079 29 7098 / 29 7098
-> sha256:34ab7071422cc07367071a746766fcd3bb19f4db9328aa08026a707157 2258 / 2258
-> extracting sha256:5b486f952f4e34451107753e544c95540c407103750618410acac079
-> extracting sha256:34ab7071422cc07367071a746766fcd3bb19f4db9328aa08026a707157
-> extracting sha256:487697150f0056a29521874e757ab1114f0ea30154e702a58c1a6e53f39
-> extracting sha256:5c080f5f90a76761345407c08c5e1da28abc784762ca62156638a089f25b
-> extracting sha256:00c72157fac180bc34216a58c420275de07675001157c642b0b0c7005e5
-> extracting sha256:00c72157fac180bc34216a58c420275de07675001157c642b0b0c7005e5
-> extracting sha256:347adda7999af310fbb09f621ed071802e96223efbba049f54f0f4a09e
-> extracting sha256:72260a0a0b10c7c2080530a23c1eac7293392990a54a5c415120a0f8
-> extracting sha256:e4805c58575c11bad1c29f653759f6195a5c16174e240077af940465211
-> extracting sha256:e084c55a5a8d21149f0bc67d11ae38892d2150aac098a069cda0b29188
-> extracting sha256:3a2c3e28c7f6a80ff4158092146c83040770376705c36a2767260400400a
-> extracting sha256:f16ef03632036203fc595a08a22974f85e091fad06409942e0880029e85f
-> extracting sha256:4f40b706bf54461c4a0271aeb0b9d61eb0d557708a6d7508dc308ac1
-> [internal] load local context
-> -- transferring context 1758
-> [2/3] RUN docker-php-ext-install pdo pdo_mysql
-> [3/3] COPY --chown=root: /var/www/html/
-> exporting to image
-> exporting layers
-> exporting manifest sha256:845f587e40834a0b70d42b7c0e2f67de584a08672386150f19997e000005014
-> exporting image
-> exporting attestation manifest sha256:600c6204918705dbf00ac42a3c320f12c2a3404330db05a515bb7700f81ab1
-> exporting manifest list sha256:f0b354aa0347d2f3c0b262539c0e2099f671f00a1f3aaf7500066d7546c9
-> naming to docker.io/library/taller-docker-app:latest
-> pushing to docker.io/library/taller-docker-app:latest
-> resolving provenance for metadata file
[*] build [*]
[*] Image taller-docker-app built
```

Figura 2.1: docker compose build

2.2. Ejecución de contenedores

Una vez construidas las imágenes, se procedió a levantar el stack completo de servicios. El comando `docker compose up -d` despliega la aplicación y la base de datos en una red virtual aislada, permitiendo la comunicación interna entre ellas.

```
root@debian:~/taller-docker# docker compose up -d
[+] up 18/18
  Image mysql:8.0                      Pulled
  Network taller-docker_crud_network    Created
  Volume taller-docker_db_data          Created
  Container mysql_crud_db               Started
  Container php_crud_app                Started
root@debian:~/taller-docker#
```

Figura 2.2: docker compose up -d

2.3. Verificación de contenedores activos

Tras ejecutar el despliegue del stack mediante el comando `docker compose up -d`, se utilizó el comando `docker ps` para validar el estado de los servicios. Como se aprecia en la figura 2.3, ambos contenedores (`php_crud_app` y `mysql_crud_db`) se encuentran en estado Up, confirmando que el entorno web y la base de datos están operando en sus puertos asignados y dentro de la red virtual definida.

```
root@debian:~/taller-docker# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                                                                 NAMES
5794916dd1e6   taller-docker-app  "docker-php-entrypoi..."  5 minutes ago  Up 5 minutes  0.0.0.0:8080->80/tcp, [::]:8080->80/tcp  php_crud_app
74f8ac4a2da9   mysql:8.0       "docker-entrypoint.s..."  5 minutes ago  Up 5 minutes  0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp, 33060/tcp  mysql_crud_db
```

Figura 2.3: docker ps

2.4. Verificación de logs y red

Para asegurar que el servicio de base de datos **MySQL** haya iniciado correctamente, se inspeccionaron los logs del contenedor mediante `docker logs mysql_crud_db`. Esto permite verificar la inicialización de los archivos de datos y la apertura del puerto 3306.

```
2026-05-31T05:25:03.679665Z 0 [System] [MY-010931] [Server] /usr/sbin/mysqld: ready for connections. Version: '8.0.46' socket: '/var/run/mysqld/mysqld.sock' port: 3306 MySQL Community Server - GPL.
```

Figura 2.4: Caption

2.5. Prueba funcional

Finalmente, se accedió a la interfaz web mediante el navegador. Se verificó que la aplicación permite realizar las operaciones **CRUD** (Crear, Leer, Modificar y Borrar) contra la base de datos persistente

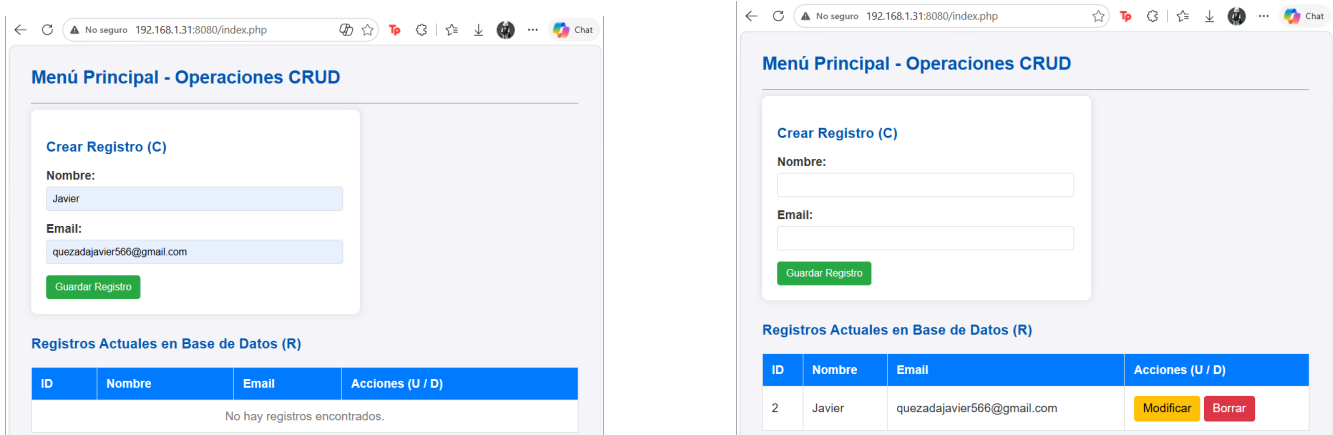


Figura 2.5: Descripción de ambas imágenes.

Capítulo 3

Conclusión

En el presente informe, se logró llevar a cabo la implementación exitosa de una aplicación web funcional utilizando contenedores. A través de este ejercicio, consolidamos el uso de **Docker** como herramienta para simplificar el despliegue de proyectos, permitiendo que la aplicación y su base de datos se conecten de forma organizada y eficiente.

Además, confirmamos que mediante el uso de unidades de almacenamiento persistente garantizamos la integridad de la información, evitando cualquier pérdida de datos al reiniciar los servicios. En resumen, este trabajo nos permitió integrar conocimientos técnicos sobre arquitectura de servicios web, logrando un sistema estable, fácil de implementar y con todas las operaciones de gestión de información plenamente operativas.